

[← ALL ARTICLES](#)

JUNE 30, 2026

The State of Trusted Open Source: June 2026

**SAM KATZEN** DIRECTOR, PRODUCT MARKETING + 1 OTHER

When we first started tracking CVE data in [The State of Trusted Open Source](#) in December 2025, the software supply chain security landscape was different. We remediated 154 CVEs across all our image projects.

[Last quarter](#), that number increased to 377 unique CVEs, as AI-assisted software development and vulnerability discovery hit the mainstream.

Fast forwarding to now, we've once again seen a major jump to 886 distinct CVEs remediated from March-May. Across all our CVE instances, over 63% were high-severity, which is a 13% increase quarter-over-quarter. AI-assisted software development is here to stay, and it is changing the way that both attackers and defenders engage with the software world. AI is helping engineers build and ship faster than ever before, but it is also enabling attackers to launch highly sophisticated, dangerous software supply chain attacks at lower cost and with fewer resources than ever before.

Taken together, these trends are creating a software ecosystem that is becoming more transparent while also becoming more complex. Organizations have more visibility into software risk than ever before. They also have more findings to evaluate, prioritize, and remediate.

Before diving into the numbers for this edition of *The State of Trusted Open Source*, it's important to explain how we perform this analysis. We examined over 2,400 unique container image projects, 18,016 total vulnerability instances, and 886 unique CVEs from March 1, 2026, through May 31, 2026. When we use terms like "top 20 projects" and "long tail projects" (as defined by images outside of the top 20), we're referring to real usage patterns observed

across our customer portfolio and in production pulls. All usage data here is anonymous across our customer base.

Key takeaways

Vulnerability discovery continues to accelerate. We observed 18,016 vulnerability instances across 886 distinct CVEs, with high-severity vulnerabilities accounting for 63.1% of all observed instances (a 13% increase quarter-over-quarter). AI-assisted security research and increasingly sophisticated analysis tools are giving organizations greater visibility into software risk across their environments.

The software supply chain continues to consolidate around a small number of foundational projects. Python remains the most widely used container image, appearing in 65.6% of analyzed customer environments, followed by Node at 60.6% and NGINX at 44.2%. Despite rapid advances in AI-assisted development, the technologies organizations build on have remained remarkably consistent.

The most widely used libraries are the infrastructure behind modern software. Across Java, JavaScript, and Python, the most common libraries are networking, packaging, logging, serialization, and utility projects that quietly underpin thousands of applications. In Python alone, urllib3 appears in 46.2% of environments, certifi in 44.9%, and requests and setuptools in 42.3%, illustrating how a relatively small set of foundational libraries supports a large share of the software ecosystem.

The long tail continues to define software supply chain risk. While the most widely used projects receive significant attention from maintainers and security researchers, 97.0% of all observed vulnerability instances occurred outside the top 20 projects. Organizations need visibility across their full dependency graph, not just the technologies they interact with directly.

Usage

The software supply chain is stabilizing around common foundations

The most popular image projects

1.  python	65.6%	11.  busybox	23.1%
2.  node	60.6%	12.  jre	22.2%
3.  nginx	44.2%	13.  ingress-nginx-controller	21.7%
4.  chainguard-base	34.7%	14.  cert-manager-controller	20.2%
5.  go	33.6%	15.  argocd	19.7%
6.  redis	29.5%	16.  cert-manager-webhook	19.5%
7.  postgres	26.9%	17.  cert-manager-cainjector	19.4%
8.  grafana	26.4%	18.  kube-state-metrics	19.1%
9.  jdk	25.8%	19.  kubectl	18.8%
10.  prometheus	24.1%	20.  prometheus-node-exporter	18.8%

 Chainguard

Python remains the most widely used image in our dataset, appearing in 65.6% of analyzed customer environments. Node follows closely behind at 60.6%, while NGINX appears in 44.2% of environments. Chainguard Base, Go, Redis, PostgreSQL, Grafana, JDK, Prometheus, BusyBox, JRE, ingress-nginx-controller, cert-manager, ArgoCD, kube-state-metrics, kubectl, and prometheus-node-exporter round out the top projects.

It might seem counterintuitive, but what's most interesting about this data is the consistency of these rankings over time.

Across multiple editions of this report, the software supply chain has continued to depend on the same foundational technologies. Organizations may adopt new frameworks, AI tooling, and development workflows, but they continue to build on a familiar infrastructure layer composed of language runtimes, databases, observability platforms, networking components, and deployment tooling.

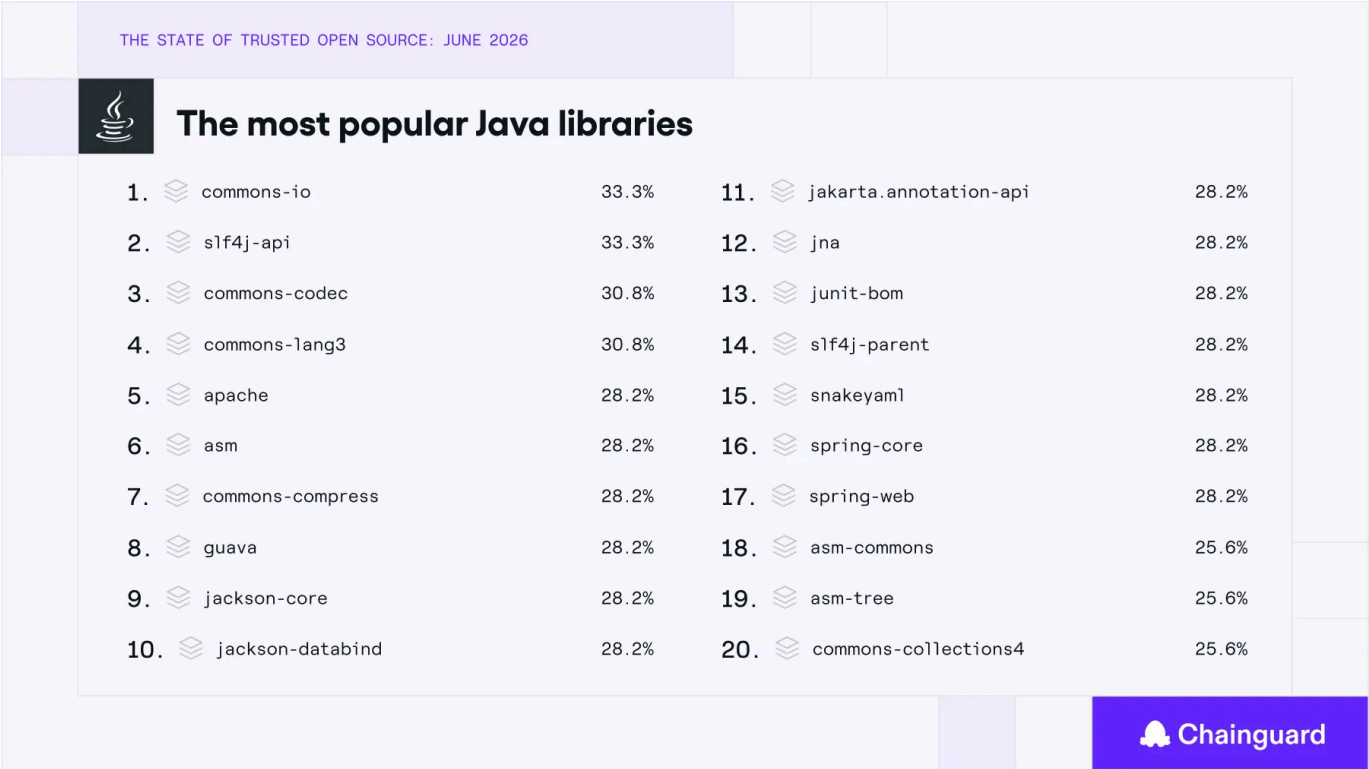
This suggests that AI is having a greater impact on software production than software foundations. Developers are creating more applications, automating more workflows, and shipping software faster, but they're doing so on top of the same trusted technologies that have powered modern software for years. Python and PostgreSQL continue to see heavy usage, with PostgreSQL climbing four spaces up the charts into the top 10 this quarter.

That concentration creates both opportunity and responsibility. The security of widely adopted projects can benefit thousands of downstream applications. Vulnerabilities affecting those projects can have equally broad consequences.

Libraries: The hidden foundation of modern software

This edition of *The State of Trusted Open Source* is the first to include [Chainguard Libraries](#) data alongside [Chainguard Containers](#). While container images provide a view into the infrastructure organizations deploy, library usage reveals the dependencies that quietly support modern applications.

Java Libraries



The most common Java libraries are overwhelmingly infrastructure-focused. Commons IO and SLF4J API each appear in 33.3% of analyzed Java environments, followed closely by Commons Codec and Commons Lang3. Other commonly used projects include Guava, Jackson Core, Jackson Databind, JNA, Spring Core, Spring Web, and SnakeYAML.

These libraries rarely receive the same attention as application frameworks, yet they play critical roles in logging, serialization, file handling, configuration management, and application connectivity. Their prevalence reflects the maturity of the Java ecosystem and the degree to which enterprise development relies on a common set of utility layers.

JavaScript Libraries

JS

The most popular JavaScript libraries

1.  lodash	36.8%	11.  jakarta.annotation-api	30.3%
2.  slf4j-api	30.3%	12.  jna	30.3%
3.  commons-codec	30.3%	13.  junit-bom	30.3%
4.  commons-lang3	30.3%	14.  slf4j-parent	30.3%
5.  apache	30.3%	15.  snakeyaml	30.3%
6.  asm	30.3%	16.  spring-core	30.3%
7.  commons-compress	30.3%	17.  spring-web	28.9%
8.  guava	30.3%	18.  asm-commons	28.9%
9.  jackson-core	30.3%	19.  asm-tree	28.9%
10.  jackson-databind	30.3%	20.  commons-collections4	28.9%

 Chainguard

Lodash is the most widely used JavaScript library, appearing in 36.8% of analyzed JavaScript environments.

Beyond Lodash, many of the popular packages are utility dependencies that most developers never interact with directly. Projects such as debug, get-intrinsic, hasown, minimatch, form-data, and related packages frequently appear as transitive dependencies within larger applications. This highlights an important characteristic of the modern JavaScript ecosystem: the software supply chain often extends far beyond the packages developers intentionally select. Applications increasingly depend on layers of supporting packages that provide foundational functionality throughout the ecosystem.

Python Libraries



The most popular Python libraries

1. urllib3	46.2%	11. platformdirs	38.5%
2. certifi	44.9%	12. typing_extensions	38.5%
3. requests	42.3%	13. pathspec	37.2%
4. setuptools	42.3%	14. anyio	35.9%
5. idna	39.7%	15. botocore	35.9%
6. packaging	39.7%	16. click	35.9%
7. pygments	39.7%	17. cryptography	35.9%
8. cffi	38.5%	18. markdown_it_py	35.9%
9. charset_normalizer	38.5%	19. pluggy	35.9%
10. filelock	38.5%	20. pycparser	35.9%

Chainguard

Python's most widely used libraries reveal how the language has evolved beyond its traditional roles.

urllib3 appears in 46.2% of analyzed Python environments, followed by certifi at 44.9%. Requests and setuptools each appear in more than 42% of environments, while idna, packaging, cryptography, charset_normalizer, filelock, platformdirs, typing_extensions, pygments, and cffi also rank among the most widely used projects.

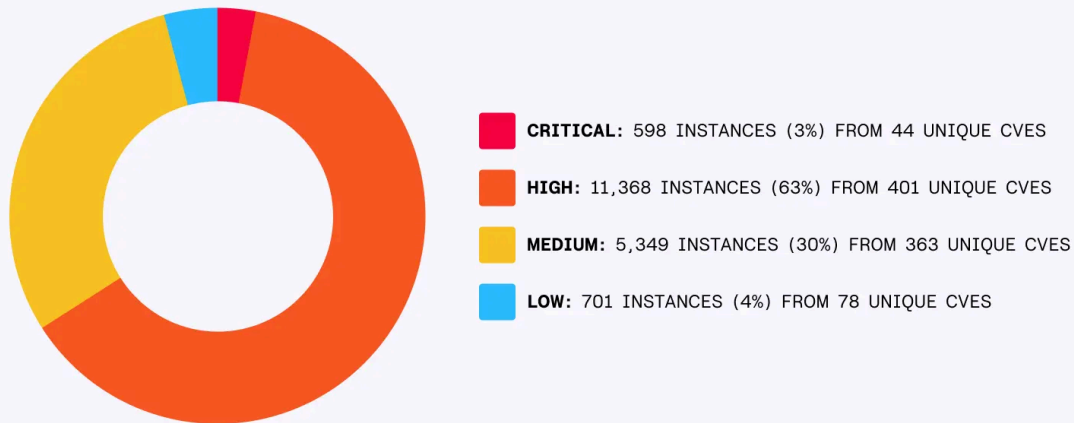
Most of these libraries are focused on networking, package management, security, compatibility, and software distribution. This reflects Python's growing role as a language for connecting systems, managing infrastructure, automating workflows, supporting AI applications, and integrating services across environments.

Across all three ecosystems, the most widely adopted libraries are foundational infrastructure projects. They may not be the most visible components of modern software, but they provide the building blocks that countless applications depend upon.

CVEs

More vulnerabilities, more visibility

Vulnerability breakdown by severity across all projects



This quarter, we observed 18,016 vulnerability instances across 886 distinct CVEs.

High-severity vulnerabilities account for 11,368 vulnerability instances, representing 63.1% of the total dataset (a 13.1% increase quarter-over-quarter). Medium-severity vulnerabilities account for 5,349 instances, while critical and low-severity vulnerabilities represent relatively small portions of overall findings.

At first glance, these numbers may appear concerning, but with context, they tell a more nuanced story.

The software industry is becoming increasingly effective at identifying and documenting vulnerabilities. Security researchers have access to more sophisticated analysis tools. Automated scanning continues to improve. AI-assisted research is helping uncover weaknesses that previously may have gone unnoticed. As a result, organizations are seeing more findings across their software environments.

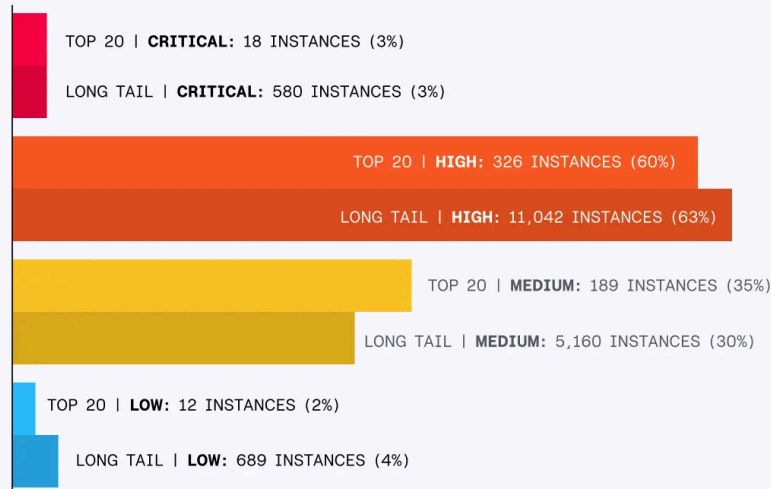
In many cases, these findings reflect an ecosystem that is becoming more transparent. Vulnerabilities that are identified can be prioritized, remediated, and monitored. Vulnerabilities that remain undiscovered cannot.

The challenge facing security teams is increasingly one of prioritization. Visibility has improved dramatically. The operational burden now lies in determining which findings deserve immediate attention and which can be addressed through longer-term remediation efforts; something that has become increasingly challenging in a world where frontier models threaten to chain low-severity vulnerabilities into serious attacks.

The long tail remains the defining challenge

THE STATE OF TRUSTED OPEN SOURCE: JUNE 2026

Top 20 vs. long tail: Number of CVE instances



 ChainGuard

One of the most consistent findings across recent editions of this report is the importance of the long tail.

This quarter, only 545 vulnerability instances occurred within the top 20 projects. The remaining 17,471 vulnerability instances occurred outside the top 20, accounting for 97% of all observed vulnerability instances.

This finding reinforces a reality that many security teams encounter every day: the projects that receive the most attention aren't the only projects that matter.

With many eyes, all bugs are shallow. Widely adopted technologies benefit from large maintainer communities, significant user bases, commercial support, and ongoing security research. Smaller dependencies often receive less visibility despite playing important roles within production applications.

As software ecosystems continue to grow, organizations increasingly need visibility beyond top-level dependencies. Effective software supply chain security requires understanding the complete set of components that make up modern applications, including the many libraries and supporting projects that exist further down dependency chains.

Compliance and trusted foundations

The most popular FIPS image projects

1.  python-fips	54.5%	11.  istio-proxy-fips	25.9%
2.  node-fips	49.6%	12.  postgres-fips	25.2%
3.  nginx-fips	45.5%	13.  cert-manager-cainjector-fips	25.2%
4.  chainguard-base-fips	32.7%	14.  cert-manager-webhook-fips	25.2%
5.  go-fips	32.3%	15.  busybox-fips	23.7%
6.  grafana-fips	29.3%	16.  kube-state-metrics-fips	23.3%
7.  redis-fips	28.6%	17.  jdk-fips	23.3%
8.  istio-pilot-fips	27.8%	18.  prometheus-node-exporter-fips	21.4%
9.  prometheus-fips	26.7%	19.  argocd-fips	21.4%
10.  cert-manager-controller-fips	26.3%	20.  glibc-openssl-fips	20.7%



Compliance-focused environments continue to rely on many of the same technologies that dominate broader software usage. [Federal Information Processing Standards](#) (FIPS) compliance is a basic requirement for any organization working with the U.S. government, handling sensitive information, or operating in regulated industries like healthcare and finance. As software supply chains grow more complex and government scrutiny of cybersecurity intensifies, FIPS compliance has become non-negotiable for teams building secure systems or serving federal markets.

Python FIPS appears in 54.5% of analyzed FIPS customer environments, followed by Node FIPS at 49.6% and NGINX FIPS at 45.5%. Chainguard Base FIPS, Go FIPS, Grafana FIPS, Redis FIPS, Istio, Prometheus, cert-manager, PostgreSQL, ArgoCD, and related infrastructure projects also rank among the most widely used images.

The similarity between FIPS and non-FIPS usage patterns suggests that compliance is becoming increasingly integrated into standard platform engineering practices.

Organizations are moving away from maintaining separate technology stacks for compliance-sensitive workloads. Instead, they are applying consistent security, compliance, and trust requirements across the software foundations that support their broader environments.

As vulnerability discovery accelerates and software supply chains become more complex, organizations increasingly benefit from standardizing on trusted foundations that can simultaneously support security, compliance, and operational requirements.

Volume and discovery continue to increase

This quarter's data points to a software ecosystem that is simultaneously stable and rapidly evolving.

The foundational technologies organizations rely on have remained remarkably consistent. Python, Node.js, NGINX, PostgreSQL, Redis, Grafana, Prometheus, and a common collection of supporting libraries continue to power a significant portion of modern software development.

At the same time, AI-assisted development is increasing software volume, while AI-assisted security research is increasing vulnerability discovery. Organizations have more visibility into software risk than ever before.

The challenge facing security teams is no longer simply finding vulnerabilities. They have to understand which vulnerabilities matter, manage an increasingly complex long tail of dependencies, and maintain trust in the foundational projects that support modern software.

Open source software supports an enormous amount of innovation. Understanding and securing open source remains one of the most important challenges in modern software security. A challenge that Chainguard is built for.

Ready to learn more about how Chainguard can protect your open source artifacts? [Get in touch](#) with our team today.

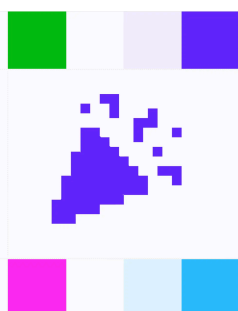


SHARE THIS ARTICLE

Related articles



What five years of Chainguard have taught us



OPEN SOURCE

OCT 1, 2026

Growing up the hard way



OPEN SOURCE

JUL 22, 2026

Growing up the hard way

What five years of Chainguard have taught us



DAN LORENC

Want to learn more about Chainguard?

Contact us 

The trusted source for open source

Talk to an expert 



PRODUCTS

Chainguard
Containers

Chainguard Libraries

Chainguard VMs

Chainguard OS
Packages

Chainguard Actions

Chainguard Agent
Skills

Images Directory

Chainguard Factory

Integrations

Pricing

SOLUTIONS

FedRAMP

PCI DSS

CMMC 2.0

SOC 2

AI Security

AI Threat Protection

Golden Images

CVE Remediation

Public Sector

Technology

Financial Services

RESOURCES

Events & Webinars

Supply Chain Security 101Blog

Chainguard Courses

Documentation

Trust Center

Chainguard Slack
Community

CUSTOMERS

Customer Stories

Chainguard Reviews

COMPANY

About Us

Open Source
Leadership

Partners

Newsroom

Compare

Careers

Legal

Support



©2026 Chainguard. All Rights Reserved.

[Privacy](#) | [Terms](#)